CSE276 · ARTIFICIAL INTELLIGENCE FOUNDATIONS

UNIT 5

Generative AI and Responsible Artificial Intelligence

How large language models work, how to get useful output from them - and the obligations that come with deploying systems that affect people.

Generative AI • Large Language Models • Prompt Engineering • RAG • Hallucination • Explainable AI • Fairness & Bias • Privacy · Governance · Ethics

School of Computer Science & Engineering

Session 2026-27 · L:2 T:0 P:2 · Credits: 3

What This Unit Covers - Two Halves

Part A is capability. Part B is the obligation that comes with it.

- PART A - Generative Artificial Intelligence
  - Generative AI concepts and their applications
  - Large Language Models
  - Prompt engineering fundamentals and prompt design techniques
  - Retrieval-Augmented Generation (RAG)
  - Hallucination in LLMs and mitigation strategies
- PART B - Responsible and Explainable AI
  - Explainable Artificial Intelligence
  - Fairness, bias and transparency
  - Privacy and security
  - AI governance, ethical and societal implications

Generate
LLMs and their applications

Control
prompting and RAG

Verify
hallucination and explainability

Govern
fairness, privacy, ethics

01

SECTION

Generative AI and Large Language Models

IN THIS SECTION

- Discriminative versus generative models
- What a large language model actually does
- Why next-token prediction produces so much
- Applications and limits

Discriminative vs Generative Models

One draws a line between categories. The other learns what the data looks like well enough to produce more of it.

	Discriminative	Generative
What it learns	The boundary between classes: P(y | x)	How the data itself is distributed: P(x) or P(x, y)
Question answered	Given this input, which class?	What does data of this kind look like?
Output	A label or a number	New data - text, image, audio, code
Examples	Logistic regression, SVM, most classifiers	LLMs, diffusion models, GANs, VAEs
Training signal	Usually labels	Usually the data itself
Typical use	Decide, classify, predict	Draft, summarise, translate, create

What a Large Language Model Actually Does

KEY CONCEPT · One deceptively simple objective

The training objective given a sequence of text, predict the next token. That is essentially all of it.

A token a chunk of text - roughly a word or part of one. Text is split into tokens before anything else happens.

Self-supervised the label is simply the next token in the existing text, so no human annotation is required. This is why training can use enormous corpora.

The architecture a transformer. Attention lets every position consider every other, which is what captures long-range context.

Why so much emerges to predict the next token well across a huge range of text, the model must implicitly capture grammar, facts, style and some reasoning patterns.

What it does NOT have a database of facts it looks things up in, a notion of truth, or awareness of what it does not know.

Consequences you should be able to state

- Fluency and correctness are separate - it optimises the first
- It cannot cite a source unless the source is in its input
- Its knowledge stops at its training cut-off

Applications of Generative AI

The common pattern: strongest where output is cheap to verify and a human stays in the loop.

TEXT

Drafting and transformation

- Summarising long documents
- Translation and rewriting for an audience
- Drafting where a human reviews the output

CODE

Programming assistance

- Completion, explanation and test generation
- Translating between languages
- Strongest where the output is easy to verify

IMAGES AND AUDIO

Synthesis

- Illustration, design variants, synthetic training data
- Speech synthesis and transcription
- Raises provenance and consent questions

KNOWLEDGE WORK

Retrieval and assistance

- Question answering grounded in your own documents
- Triage, routing and first-draft responses
- Best paired with retrieval - section 03

02

SECTION

Prompt Engineering

IN THIS SECTION

- What a prompt actually controls
- Zero-shot, few-shot and chain-of-thought
- Prompt design techniques that generalise
- Why prompt claims need evidence

Prompt Design Techniques

Six techniques that transfer across models and tasks.

TECHNIQUE 1

Be specific about the task

- State the audience, format and length
- 'Explain to a first-year student in three bullets'
- Vagueness in, vagueness out

TECHNIQUE 2

Assign a role

- 'You are an experienced examiner marking this answer'
- Sets vocabulary and level
- Cheap, and often surprisingly effective

TECHNIQUE 3

Few-shot examples

- Show two or three worked input-output pairs
- The model infers the pattern and the format
- Usually beats describing the format in words

TECHNIQUE 4

Chain-of-thought

- 'Think step by step, then give a final answer'
- Helps on multi-step reasoning and arithmetic
- Costs more tokens and more time

TECHNIQUE 5

Constrain the output

- Specify JSON, a table, or exactly three bullets
- Makes the output machine-checkable
- Essential when another program consumes it

TECHNIQUE 6

Give it permission to refuse

- 'If the context does not contain the answer, say so'
- The single most effective anti-hallucination instruction
- Turns silence into an acceptable output

Evaluate Prompts, Do Not Just Believe Them

EXAM TIP · The habit that separates engineering from folklore

The problem prompt advice circulates as folklore - 'this phrasing works better' - with no evidence attached.

Why that fails results vary by model, by task and by run. A technique that helps one model may do nothing for another.

What to do instead define a small set of test tasks, and a RUBRIC you can compute - length, structure, presence of required facts, correct refusal.

Then compare run every prompt strategy against the same tasks under the same conditions, and record the scores.

Report the claim WITH the rubric 'chain-of-thought scored 0.81 against 0.62 on this three-part rubric across ten tasks' is checkable. 'Chain-of-thought is better' is not.

Know your rubric's limits an automatic rubric cannot detect a confident factual error, which is exactly why humans still review.

DESIGN ONE What would you measure to test whether a prompt reduces hallucination?

03

SECTION

Hallucination and Retrieval-Augmented Generation

IN THIS SECTION

- What hallucination is, and why it happens
- Retrieval-Augmented Generation
- Grounding and citation
- Mitigation strategies that actually work

Hallucination - Confident, Fluent and Wrong

COMMON PITFALL · Not a bug to be patched, but a consequence of the objective

What it is the model produces text that is fluent, plausible and factually wrong - often with citations, dates or names that do not exist.

Why it happens the training objective rewards PLAUSIBLE continuations, not true ones. Nothing in it distinguishes a correct fact from a convincing one.

No notion of not knowing the model has no internal signal for 'I lack this information', so it produces its best guess with the same confidence as a fact.

Where it is worst specific facts - numbers, dates, citations, names, quotations - and any topic sparsely represented in training.

Why fluency makes it dangerous a wrong answer that reads well receives less scrutiny than one that reads badly.

The mitigation that matters most ground the model in retrieved source text and require it to answer only from that.

Retrieval-Augmented Generation (RAG)

Retrieval supplies the truth; the model supplies the fluency.

01 INDEX

- Split your documents into chunks
- Store them for search
- Done once, in advance

02 RETRIEVE

- Take the user's question
- Find the most relevant chunks
- Keyword search or embeddings

03 AUGMENT

- Insert those chunks into the prompt as CONTEXT
- Instruct: answer only from this context

04 GENERATE

- The model writes the answer
- Fluency from the model, facts from your documents

05 CITE

- Return the source chunks alongside the answer
- The user can verify it
- This is what makes it auditable

CONSIDER What happens if retrieval returns nothing relevant?

Mitigation Strategies for Hallucination

EXAM TIP · In rough order of effectiveness

1 - **Ground with retrieval** give the model the source text and instruct it to answer only from that. The single most effective measure.

2 - **Require citation** make the model quote or reference the supporting passage, so an unsupported claim is visible.

3 - **Permit refusal explicitly** 'if the context does not contain the answer, say you do not know.' Without this, the model will invent something.

4 - **Constrain the output format** structured output is easier to validate automatically than free prose.

5 - **Verify externally** check facts against a database or a calculator rather than trusting the model's arithmetic.

6 - **Keep a human in the loop** for any consequential decision. None of the above eliminates hallucination - they reduce and expose it.

What does NOT reliably work

- Asking the model 'are you sure?' - it will often agree either way
- Assuming a larger model has removed the problem
- Trusting confidence in the wording as a signal of correctness

CHECK YOUR UNDERSTANDING

Quick Check - Generative AI

UNIT 5

QUICK CHECK

A RAG system returns an answer that is fluent and confident, but the retrieved context did not contain the relevant fact. What went wrong, and where?

A. The language model failed - it should be retrained

B. Retrieval failed, and the prompt did not require refusal

C. The user's question was badly worded

D. RAG cannot handle factual questions

ANSWER: B There are two separate failures. Retrieval did not find the right passage, and the prompt did not instruct the model to say it could not answer from the given context. Fixing RAG usually means fixing retrieval, not the model - and every RAG prompt should permit refusal.

04

SECTION

Explainable AI

IN THIS SECTION

- Why explainability is a requirement, not a nicety
- Global versus local explanations
- LIME and SHAP
- Interpretable by design

Global vs Local Explanations

A regulator does not ask what your model uses in general. They ask why this person was refused.

Global explanation

The question how does this model behave in general?

Typical output feature importances, partial dependence plots, the overall decision logic.

Used by the team building and validating the model.

Answers which features does it rely on? Is it using something it should not?

Limitation it says nothing about any particular decision.

Local explanation

The question why did the model decide THIS case this way?

Typical output the contribution of each feature to one specific prediction.

Used by the affected person, a regulator, an auditor.

Answers why was MY loan refused?

Why it matters legally in lending and similar domains an adverse decision must be explained to the individual.

LIME and SHAP - The Two Standard Techniques

KEY CONCEPT · **Both explain individual predictions of any model**

The shared idea treat the model as a black box, probe it, and build a simple local approximation of its behaviour.

LIME perturb the input many times, observe how the prediction changes, and fit a simple interpretable model to that local behaviour.

LIME's strength fast, intuitive, and works on text, images and tabular data.

LIME's weakness the explanation can change between runs, because the perturbations are random.

SHAP assigns each feature a contribution using a result from cooperative game theory, giving consistent and additive attributions.

SHAP's trade stronger theoretical guarantees, at a considerably higher computational cost.

Interpretable by design

- An alternative: use a model that is readable in the first place
- A shallow decision tree or a linear model needs no post-hoc explanation
- Often the right answer in regulated domains, even at some accuracy cost

05

SECTION

Fairness, Privacy and Governance

IN THIS SECTION

- Where bias actually comes from
- Why removing a protected attribute is not enough
- Privacy and security
- AI governance and societal implications

Where Algorithmic Bias Comes From

Bias rarely comes from a biased line of code. It comes from the data and the deployment.

SOURCE 1

Historical bias

- The training labels record past human decisions
- If those were biased, the model learns the bias
- The data is accurate AND the outcome unjust

SOURCE 2

Representation bias

- Some groups are under-represented in the data
- The model performs worse for them
- Accuracy looks fine in aggregate

SOURCE 3

Measurement bias

- The proxy you measure is not what you mean
- 'Arrests' is not 'crimes committed'
- The gap differs between groups

SOURCE 4

Deployment bias

- The system is used differently from how it was designed
- A screening tool becomes a final decision
- The model never changed; the use did

Why Removing a Protected Attribute Is Not Enough

COMMON PITFALL · The most common mistaken defence

The claim 'our model is fair - we never gave it gender, caste or religion.'

Why it fails other features correlate with the excluded one. Postcode, income, education, device type and purchase history can all carry it.

The name PROXY VARIABLES. The model reconstructs the attribute without ever being given it.

How to check test whether the protected attribute is predictable from your model's inputs or outputs. If it is, the information is present.

Measure the disparity compare selection rate and true positive rate across groups, and report the gaps.

Then decide deliberately reweight, drop features, adjust thresholds, or accept the disparity with a documented justification - but never claim fairness by omission.

An impossibility worth knowing

- Several reasonable definitions of fairness exist
- It is proven they cannot all be satisfied at once, except in degenerate cases
- So choosing a definition is a value judgement, not a technical one

Privacy, Security and Governance

Six areas, one question underneath all of them: who is accountable when it goes wrong?

PRIVACY

Data minimisation and consent

- Collect only what the purpose requires
- Consent given for one purpose does not cover another
- Anonymisation is weaker than people assume

SECURITY

Model-specific attacks

- Adversarial inputs crafted to cause misclassification
- Data poisoning during training
- Prompt injection against LLM applications

GOVERNANCE

Accountability structures

- Who is answerable when the system is wrong?
- Documentation: model cards, data statements
- Audit trails and the right to appeal

TRANSPARENCY

Disclosure

- Telling people a decision was automated
- Publishing intended use and known limitations
- Stating what the system must not be used for

SOCIETAL

Wider implications

- Labour displacement and skill change
- Concentration of capability in few organisations
- Environmental cost of large-scale training

THE HABIT

Ask before deploying

- Who could this harm, and how would we detect it?
- What is the appeal route for someone affected?
- What must this system never be used for?

Activity: Build It, Then Audit It

GROUP WORK · 12 minutes

In groups of 3-4, design a generative AI application and then act as its critic.

1

Choose an application

- A student query assistant for your department
- An assistant that drafts replies to customer complaints
- A tool that summarises medical discharge notes
- Or propose your own

2

Design the system

- Would you use RAG? What would you index?
- Write the key instruction that permits refusal
- How would you evaluate the output?

3

Audit it

- Where could it hallucinate, and what would that cost?
- Who could it treat unfairly, and how would you check?
- What privacy concern does your data source raise?

4

Govern it

- Who is accountable when it is wrong?
- What is the appeal route?
- State one use you would explicitly forbid

DELIVERABLE One speaker per group: 60 seconds on your application, your strongest safeguard, and the risk you could not fully mitigate.

UNIT V RECAP

Unit V - The Eight Ideas Worth Remembering

UNIT 5

1. **Generative models learn what data looks like**
   Discriminative models learn a boundary between classes; generative models learn the distribution well enough to produce new samples.

2. **An LLM predicts the next token**
   That single self-supervised objective, at scale, is what produces grammar, style, facts and some reasoning as by-products.

3. **Fluency and correctness are separate**
   The objective rewards plausible continuations, not true ones, which is precisely why hallucination happens.

4. **Prompt design is specificity, examples and permission to refuse**
   Role, format constraints, few-shot examples and chain-of-thought all transfer across models.

5. **RAG grounds generation in retrieved text**
   Retrieval supplies the facts, the model supplies the fluency, and citations make the result auditable.

6. **Explanations are global or local**
   Global describes the model; local justifies one decision - and it is local explanations that regulators require.

7. **Bias comes from data and deployment, not from biased code**
   Historical, representation, measurement and deployment bias are the four sources worth naming.

8. **Removing a protected attribute does not remove it**
   Proxy variables reconstruct it. Measure the disparity, document it, and decide deliberately.

NEXT UP Unit VI - The AI Ecosystem and Future Trends: the Python ecosystem, TensorFlow and PyTorch, Hugging Face, Ollama and LangChain, vector databases, and where the field is heading.